

CSE 312 — İşletim Sistemleri

FILE SYSTEMS

Kapsamlı Türkçe Ders Notu

Ders: CSE 312 İşletim Sistemleri

Dönem: Spring 2026

Kapsam: Lecture 07 — File Systems (Slides 1–136)

Hazırlayan: Kapsamlı Sınav Hazırlık Notu

Sürüm: 1.0

Contents

1	File Systems’e Giriş: Temel Kavramlar	5
1.1	File Nedir? File System Nedir?	5
1.2	Uzun Vadeli Bilgi Depolamanın Gereksinimleri	5
1.3	Disk’in Soyutlanması	6
2	Tasarım Kararları (Design Decisions)	6
2.1	Dosya Adlandırma (File Naming)	6
2.1.1	Uzantılar (Extensions)	6
2.2	Tasarım Kararlarının Tam Listesi	7
2.3	Dosya Yapıları (File Structures)	7
2.4	Medya Özellikleri ve Hız	8
2.5	Sürümleme (Versioning)	8
3	Metadata: Dosya Hakkındaki Veri	8
3.1	Metadata Nedir ve Neden Gereklidir?	8
3.2	Linux <code>stat</code> Yapısı	8
3.3	Metadata Nerede Saklanır?	9
4	Disk Alanı Organizasyonu ve Allocation Yöntemleri	9
4.1	Temel Kavramlar	10
4.2	Contiguous Allocation (Bitişik Yerleştirme)	10
4.2.1	Nasıl Çalışır?	10
4.3	Linked Allocation (Bağlı Liste ile Yerleştirme)	11
4.3.1	Nasıl Çalışır?	11
4.4	FAT — Linked Allocation Using a Table in Memory	11
4.4.1	Temel Fikir	11
4.5	Indexed Allocation ve I-node	13
4.5.1	Temel Fikir	13
4.5.2	I-node Yapısı	13
4.5.3	Çok Kademeli Dolaylı Bloklar (Multi-level Indirection)	13
4.6	Üç Allocation Yönteminin Karşılaştırması	15
5	Directory Yapısı ve Uygulaması	15
5.1	Hiyerarşik Directory Sistemi	15
5.1.1	Path Name Çözümleme	15
5.1.2	. ve .. Girişleri	16
5.2	Directory Entry Yapıları	16
5.2.1	Sabit Boyutlu Girişler	16
5.2.2	Değişken Uzunluklu İsimler	16
5.3	Shared Files (Hard Links ve Soft Links)	16

5.3.1	Hard Link	17
5.3.2	Soft Link (Symbolic Link)	17
5.4	Directory Operasyonları	18
6	File Operasyonları: Açma'dan Kapama'ya	18
6.1	Dosya Operasyonlarına Genel Bakış	18
6.2	Dosya Açma (Open)	19
6.3	Dosya Oluşturma (Create)	19
6.4	Okuma (Read)	19
6.5	Yazma (Write)	19
6.6	Seek	20
6.7	Dosya Kapatma (Close)	20
6.8	Linkleme ve Linklemeyi Kaldırma	20
7	Free-Space Yönetimi	20
7.1	Boş Blok Takibinin Önemi	20
7.2	Bağlı Liste (Linked List of Free Blocks)	20
7.3	Bitmap (Bit Haritası)	21
7.4	Disk Kotaları (Disk Quotas)	21
8	Crash Recovery ve Journaling	21
8.1	Tutarsızlık Problemi	21
8.2	fsck — File System Consistency Check	22
8.2.1	Blok Tutarlılığı Kontrolü	22
8.3	Log-Structured File Systems ve Journaling	23
8.3.1	Modern Disk Yazma Problemleri	23
9	Virtual File System (VFS)	23
9.1	VFS Nedir ve Neden Gereklidir?	23
9.2	VFS'in Çalışma Prensibi	24
10	File System Backup	25
10.1	Yedeklemenin Amacı	25
10.2	Tasarım Kararları	25
10.3	Fiziksel Dump vs. Mantıksal Dump	25
10.4	Mantıksal Dump Algoritması	26
10.5	Geri Yükleme Algoritması	26
10.6	UNIX Dump Seviyeleri	26
11	File System Performansı: Caching ve Optimizasyon	27
11.1	Temel Sorun: Disk vs. Bellek Hızı	27
11.2	Buffer Cache	27
11.2.1	Değiştirilmiş LRU (Modified LRU)	27
11.3	Block Read-Ahead (Blok Önceden Okuma)	28

11.4 Disk Kol Hareketini Azaltma (Cylinder Groups)	28
11.5 Blok Boyutu Seçimi	28
12 File System Bileşenleri: Superblock, I-list, Data Area	28
12.1 Tipik Disk Düzeni	29
12.2 Boot Record ve Superblock	29
12.3 I-list (I-node Listesi)	29
12.4 Veri Alanı (Data Area)	29
13 UNIX V7 Dosya Sistemi — Adım Adım	29
13.1 UNIX V7 Directory Entry	29
13.2 Dosyaya Erişim: /usr/ast/mbox	30
14 Windows FAT Dosya Sistemi	30
14.1 FAT'ın Tarihçesi ve Sürümleri	30
14.2 FAT Yapısı	30
14.3 Uzun Dosya Adı Desteği (VFAT)	31
15 ISO 9660 (CD-ROM) Dosya Sistemi	31
16 Uygulamalı Mini Problemler	31
16.1 Problem 1: FAT Bellek Hesabı	31
16.2 Problem 2: I-node Hesabı — Dosya Boyutu	32
16.3 Problem 3: Yol Çözümleme Erişim Sayısı	32
16.4 Problem 4: Bitmap Boyutu	32
16.5 Problem 5: fsck Senaryo Analizi	32
17 UNIX File System Operasyonları — Kod Düzeyinde	33
17.1 Tipik Dosya İşlem Akışı	33
17.2 Rename Operasyonunun İnceliği	33
17.3 Dizin Oluşturma ve Silme	34
18 Karşılaştırma Tabloları Özeti	34
18.1 Dosya Sistemleri Karşılaştırması	34
18.2 Crash Recovery Yöntemleri Karşılaştırması	35
19 Terimler Sözlüğü (Glossary)	35
20 Sınav Cevap Şablonları	37
20.1 “Allocation yöntemlerini karşılaştırın” — Model Yanıt	37
20.2 “I-node’u açıklayın” — Model Yanıt	38
20.3 “Journaling nedir?” — Model Yanıt	38
21 Kavramsal Bağlantı Haritası	38
22 Final Revizyon: 25 Kritik Sınav Noktası	39
23 Ek: Önemli Formüller Özeti	41

1 File Systems'e Giriş: Temel Kavramlar

1.1 File Nedir? File System Nedir?

İşletim sistemi dersinin en somut ve kullanıcıya en görünür parçalarından biri **file system**'dir. Günlük bilgisayar kullanımında her işlem bir şekilde file system ile temas eder: bir belge açmak, bir program çalıştırmak, bir resim kaydetmek... Peki dosya nedir?

★ Önemli Nokta

File (Dosya): İşletim sisteminin depolama kaynakları için sunduğu soyut arayüzde **mantıksal depolama birimidir**. İki temel rolü vardır:

- **Geçici dosyalar:** Adres alanının disk üzerine uzantısı (örn. swap dosyaları).
- **Kalıcı dosyalar:** Bir programın çalışma süresinin ötesinde var olan, *non-volatile* (uçucu olmayan) bilgidir.

File System: Dosya adlarını fiziksel ortamın bir alt kümesine eşleyen **haritalama yapısıdır**. Daha geniş tanımıyla; işletim sisteminin depolama kaynaklarına yönelik soyutlama katmanıdır.

Directory (Dizin): Bir grup dosya için mantıksal "konteynerdir".

1.2 Uzun Vadeli Bilgi Depolamanın Gereksinimleri

Neden disk gibi harici depolama aygıtlarına ihtiyaç duyarız? RAM yeterli değil midir? RAM verileri kaybeder; disklere olan ihtiyaç üç temel zorunluluktan kaynaklanır:

1. **Çok büyük miktarda bilgi saklanabilmelidir.** RAM kapasitesi sınırlıdır; modern bir dijital film 2 petabayt (500.000 DVD) üretebilir. 1 petabayt'ı 1 Gbps Ethernet üzerinden aktarmak 90 gün sürer.
2. **Bilgi, onu kullanan sürecin (process) sonlanmasından sonra da yaşmalıdır.** Kalıcılık (persistence) kritiktir.
3. **Birden fazla süreç bilgiye eş zamanlı erişebilmelidir.** Paylaşımlı erişim desteği zorunludur.

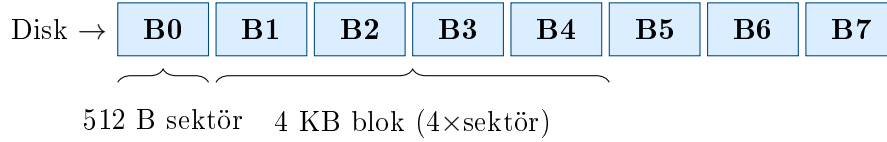
□ Sınav Yorumu

"Dosya sisteminin neden gerekli olduğunu 3 madde ile açıklayın" sorusu çok klasiktir. Yukarıdaki 3 maddeyi ezberlemek yerine *anlamak* tercih edilmelidir: kapasite, kalıcılık ve paylaşım.

1.3 Diskin Soyutlanması

Bir diski, sabit boyutlu blokların doğrusal dizisi olarak düşünebiliriz; disk bu blokları okuma ve yazma işlemlerini destekler. Bu soyutlama üç temel soruyu doğurur:

- **Bilgi nasıl bulunur?** (İsimden bloğa nasıl ulaşılır?)
- **Bir kullanıcı başkasının verisini nasıl okuyamaz?** (Erişim denetimi)
- **Hangi blokların boş olduğu nasıl bilinir?** (Free-space yönetimi)



2 Tasarım Kararları (Design Decisions)

2.1 Dosya Adlandırma (File Naming)

Farklı işletim sistemleri farklı adlandırma kuralları benimser:

OS / Sistem	Adlandırma Biçimi
UNIX/Linux	<code>/this/is/a/unix.file.name</code> — Büyük/küçük harf duyarlı (case-sensitive)
Windows	<code>q:\Windows likes\mOn0caSE</code> — Büyük/küçük harf duyarsız
Multics	<code>>Multics>uses<for>up-a-level</code> — <code>></code> hiyerarşi ayracı
VMS	<code>vms-host::device[dir.list]name.type;1</code> — Sürüm numarası içerir
DOS	8.3 biçimi: 8 karakter isim + 3 karakter uzantı

2.1.1 Uzantılar (Extensions)

- **UNIX ve Multics'te:** Çekirdek (kernel) dosya uzantısından tamamen habersizdir; uzantılar sadece birer karakter dizisidir. Ancak `make`, C derleyicisi gibi uygulamalar uzantılara anlam yükler.
- **DOS'ta:** 8.3 biçimi zorunlu; 3 karakterlik uzantı OS tarafından bazı dosya türleri için anlamlıdır.
- **Diğer sistemler:** Bazı sistemler uzantıları izin seviyeleri ile karıştırmıştır.

2.2 Tasarım Kararlarının Tam Listesi

Karar Alanı	Seenekler / Açıklama
Dosya adları	Büyük/küçük harf duyarlılığı, uzunluk, karakter seti
Yapı	Hiyerarşik mi, tek seviye mi?
Erişim denetimi	Kullanıcı/grup/diğer izinleri; ACL
Medya özellikleri	Sabit sektör, deęişken sektör, flash, WORM
Mevcut alan	Sıkışık mı, bolca mı? Hız-alan dengesi
Medya hızı	Disk erişim süreleri tasarımı etkiler
Sadece-okuma / Okuma-yazma	CD-ROM vs. HDD
Sürümleme	VMS sürüm numaralarını destekler
Hata kurtarma	Crash sonrası tutarlılık nasıl saęlanır?
Yapı birimi	Kayıt tabanlı, bayt tabanlı veya blok tabanlı

2.3 Dosya Yapıları (File Structures)

★ Önemli Nokta

Üç klasik dosya yapısı:

- **Byte sequence (Bayt dizisi):** UNIX/Linux'un benimsedięi model. OS içerięi yorumlamaz; tüm anlam uygulamaya bırakılır. Rastgele adreslenebilir.
- **Record sequence (Kayıt dizisi):** Eski mainframe sistemleri. Dosya sabit boyutlu kayıtlardan oluşur (ilk örnekler delikli kart veya yazıcı satırı görüntüleri).
- **Tree (Aęaç):** Kayıtlara anahtar ile erişilen hiyerarşik yapı. Veritabanı odaklı.

□ Sınav Yorumu

“UNIX dosya yapısı neden ‘byte sequence’ modelini benimsemiştir?” sorusu sık sorulur. Yanıt: *simplicity*. OS içerięi bilmek zorunda kalmaz; esneklik uygulamaya devredilir.

2.4 Medya Özellikleri ve Hız

- Modern diskler sabit 512 baytlık sektörlere sahipken eski mainframe diskleri değişken sektör boyutunu destekliyordu.
- CD ve DVD'ler blok düzeyinde yazılabilir değildir (sıralı yazma zorunluluğu).
- Flash disklerde bir blok belirli sayıda yazma sonrası yıpranır (*wear leveling* gerekir).
- WORM (Write Once Read Many) diskler: Silinebilir ama üzerine yazılamaz.

2.5 Sürümleme (Versioning)

VMS, Tenex ve TOPS-20 gibi sistemler dosya adının bir parçası olarak sürüm numarası taşır (`file.txt;3` gibi). Bu sayede bir dosyanın eski sürümlerine erişmek kolaylaşır. Plan 9 ise dosya sisteminin herhangi bir geçmiş anına pencere açma imkânı sunar.

3 Metadata: Dosya Hakkındaki Veri

3.1 Metadata Nedir ve Neden Gereklidir?

Metadata, dosyanın içeriği değil, dosya *hakkındaki* bilgidir. İşletim sistemi bir dosyayı diskte konumlandırmak, korumak ve yönetmek için bu bilgilere muhtaçtır.

★ Önemli Nokta

Temel metadata alanları:

- Disk blok adresleri (içerik nerede?)
- Dosyayı oluşturan kullanıcı (owner)
- Oluşturma / değiştirme / erişim tarihleri
- Dosya izinleri (permissions)
- Dosya boyutu
- Link sayısı (hard link count)

3.2 Linux stat Yapısı

Linux çekirdeği metadata'yı `struct stat` ile temsil eder:

Listing 1: Linux stat yapısı — temel metadata alanları

```
1 struct stat {
2     dev_t      st_dev;      /* Aygıt numarası */
3     ino_t      st_ino;      /* I-node numarası */
4     mode_t     st_mode;     /* Dosya türü ve izinler */
5     nlink_t    st_nlink;    /* Hard link sayısı */
6     uid_t      st_uid;      /* Sahibin kullanıcı ID'si */
7     gid_t      st_gid;      /* Sahibin grup ID'si */
```



```

8      off_t      st_size;      /* Toplam boyut (bayt) */
9      blksize_t  st_blksize;   /* Dosya sistemi I/O blok boyutu */
10     blkcnt_t    st_blocks;    /* Ayrılmış blok sayısı */
11     time_t      st_atime;     /* Son erişim zamanı */
12     time_t      st_mtime;     /* Son değişiklik zamanı */
13     time_t      st_ctime;     /* Son durum değişikliği zamanı */
14 };

```

△ Dikkat

`st_mtime` ve `st_ctime` arasındaki farkı bilmek önemlidir:

- `st_mtime`: Dosya **içeriği** değiştiğinde güncellenir.
- `st_ctime`: Dosya **metadata'sı** değiştiğinde güncellenir (izin değişikliği, link ekleme vb.). *Creation time* değildir!

3.3 Metadata Nerede Saklanır?

Konum	Avantajlar	Dezavantajlar
Dosyanın içinde	Taşınabilirlik sorunu yok (her şey tek yerde)	Kopyalamada metadata kaybolabilir; standart yok
Directory entry	Metadata'ya hızlı erişim	Hard link zor (kopyaları senkronize etmek gerekir); directory şişer
Ayrı yapıda (i-node)	Temiz ayrışım; hard link kolay	Ek disk erişimi gerekebilir

★ Önemli Nokta

UNIX yaklaşımı: Metadata büyük ölçüde **i-node**'da saklanır. Directory entry yalnızca *dosya adı* → *i-node numarası* eşlemesini tutar. Bu sayede *hard link* oluşturmak son derece kolaylaşır: farklı directory entry'ler aynı i-node'u gösterebilir.

Bazı modern sistemler hız için dosya türü gibi birkaç bit'i directory'de de tutarlar (ağaç gezimlerini hızlandırmak için).

4 Disk Alanı Organizasyonu ve Allocation Yöntemleri

4.1 Temel Kavramlar

- **Sektör (sector):** Diskin temel I/O birimi; genellikle 512 bayt.
- **Blok (block):** File system'ın seçtiği büyük birim; tipik olarak 4 KB (8 sektör). Dosya sistemi her zaman tam blok okur/yazar.
- **Allocation:** Bir dosyaya hangi disk bloklarının atandığını belirleme işlemi.

Σ Formül

Blok sayısı hesabı:

$$N_{\text{blok}} = \left\lceil \frac{\text{Dosya Boyutu (bayt)}}{\text{Blok Boyutu (bayt)}} \right\rceil$$

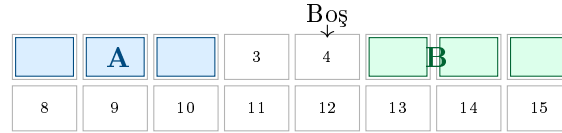
Örnek: 10 KB dosya, 4 KB bloklar $\rightarrow \lceil 10/4 \rceil = 3$ blok kullanılır.

İç parçalanma (internal fragmentation): Son blokta boş kalan yer $= 3 \times 4 \text{ KB} - 10 \text{ KB} = 2 \text{ KB}$.

4.2 Contiguous Allocation (Bitişik Yerleştirme)

4.2.1 Nasıl Çalışır?

Her dosya disk üzerinde **ardışık (contiguous) bloklar** kümesini kaplar. Directory entry yalnızca başlangıç blok numarasını ve toplam blok sayısını saklar.



Kriter	Değerlendirme	Detay
Uygulama kolaylığı	Çok kolay	Yalnızca başlangıç + uzunluk
Sıralı erişim	Çok hızlı	Bloklar yanyana; düşük seek
Rastgele erişim	Hızlı	Doğrudan blok adresi hesaplanır
Dış parçalanma	Ciddi sorun	Zamanla küçük boşluklar birikir
Dosya büyütme	Çok zor	Ardışık alan bulunamayabilir

✓ Örnek

Dış Parçalanma Senaryosu: Disk üzerinde A (3 blok), B (5 blok), C (2 blok), D (4 blok) dosyaları ard arda yerleştirilmiş olsun. B ve D silinince $5+4=9$ blok *serbest* olmasına rağmen, ayrı parçalarda kaldıkları için 7 blok isteyen yeni bir dosya yerleştirilemeyebilir. Bu dış parçalanmadır. Çözüm: disk birleştirme (defragmentation) — ama bu pahalıdır.

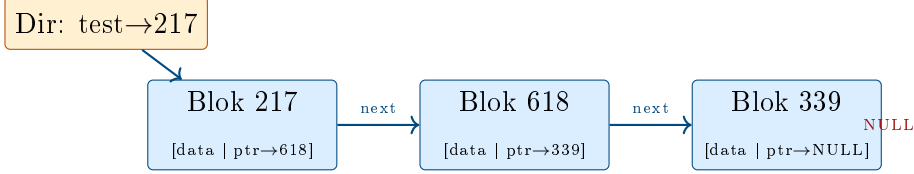
□ Sınav Yorumu

“Contiguous allocation’ın en büyük dezavantajı nedir?” → **Dış parçalanma (external fragmentation)**. “Avantajı nedir?” → **Hızlı sıralı ve rastgele erişim**. Bu iki yanıt sınav klasiğidir.

4.3 Linked Allocation (Bağlı Liste ile Yerleştirme)

4.3.1 Nasıl Çalışır?

Her dosya bir **bağlı liste** (linked list) olarak temsil edilir. Her disk bloğu, listedeki sonraki bloğun adresini taşır. Directory entry yalnızca ilk bloğun adresini saklar.



Kriter	Değerlendirme	Detay
Alan verimliliği	İyi	Dış parçalanma yok
Dosya büyütme	Kolay	Yeni blok listenin sonuna eklenir
Sıralı erişim	Orta	Pointer takibi gerekir
Rastgele erişim	Çok yavaş	N. bloğa ulaşmak için N adım seyahat
Güvenilirlik	Kırılgan	Bozuk bir pointer tüm dosyayı mahveder

4.4 FAT — Linked Allocation Using a Table in Memory

4.4.1 Temel Fikir

Linked allocation’ın zayıf rastgele erişim problemini çözmek için pointer’lar blokların içinden *çıkarılıp* ayrı bir tabloya taşınır: **File Allocation Table (FAT)**. Bu tablo RAM’de tutulur.

FAT	
0	0
1	5
2	3
3	EOF
4	7
5	2
6	EOF
7	EOF

Blok# Sonraki

Dosya A: {4,7,2,10,12}

Dosya B: {6,3,11}

FAT tüm dosyaların
zincir bilgisini
saklar. Rastgele erişimde disk
okumadan ilerlenir!

Σ Formül

FAT tablosu için gereken RAM:

$$\text{FAT RAM} = \frac{\text{Disk Boyutu}}{\text{Blok Boyutu}} \times \text{FAT entry boyutu}$$

Örnek: 200 GB disk, 1 KB blok, 4 bayt entry:

$$\frac{200 \times 10^9}{1024} \times 4 \approx 781 \text{ MB}$$

Bu son derece büyük! FAT16/FAT32 blok boyutunu artırarak bu sorunu kısmen hafifletir.

Kriter

FAT Avantajı

FAT Dezavantajı

Erişim hızı	Linked allocation'dan daha hızlı rastgele erişim (tablo RAM'de)	Uzun zincirler hâlâ traverse edilmeli
RAM kullanımı	—	Büyük diskler için çok fazla RAM gerekir
Güvenilirlik	Tablo disk başında da saklanır	FAT bozulursa felakettir
Uygulama kolaylığı	Basit veri yapısı	FAT32 max dosya boyutu 4 GB

□ Sınav Yorumu

FAT hakkında sınav soruları şunlardır:

1. “FAT için gereken bellek miktarını hesaplayın” — Yukarıdaki formülü uygulayın.

2. “FAT’ın linked allocation’a göre avantajı nedir?” → Pointer’lar RAM’de; disk okumadan traverse.
3. “FAT’ın dezavantajı nedir?” → Çok RAM tüketir; hâlâ traverse gerekmektedir.

4.5 Indexed Allocation ve I-node

4.5.1 Temel Fikir

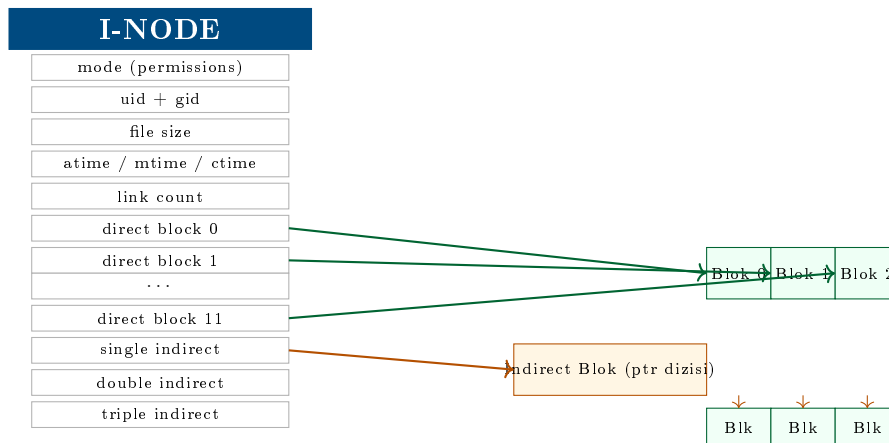
Tüm pointer’lar **index block** (veya **i-node = index node**) adı verilen tek bir yapıda toplanır. FAT’ın aksine her dosyanın kendi index bloğu vardır; tüm dosya sistemi için tek bir büyük tablo tutulmaz.

★ Önemli Nokta

I-node (Index Node):

- Dosyaya ait tüm **metadata** ve **disk blok adreslerini** içeren yapıdır.
- Yalnızca dosya **açık olduğunda** RAM’e yüklenir; kapatıldığında diske geri yazılır.
- **I-list**: Diskte tüm i-node’ların tutulduğu dizi (array).
- I-node **numarası**: Bu dizinin indeksi; yani bir i-node numarası basitçe diziye bir işaretçidir.

4.5.2 I-node Yapısı



4.5.3 Çok Kademeli Dolaylı Bloklar (Multi-level Indirection)

UNIX v7 i-node’ları şu yapıyı kullanır (4 KB blok, 4 bayt pointer varsayımıyla):

Seviye	Açıklama	Kapsanan Boyut
12 doğrudan (direct)	I-node'dan direkt blok adresi	$12 \times 4 \text{ KB} = 48 \text{ KB}$
Tek dolaylı (single indirect)	Bir blok pointer dizisi tutar	$1024 \times 4 \text{ KB} = 4 \text{ MB}$
Çift dolaylı (double indirect)	İki seviye pointer	$1024^2 \times 4 \text{ KB} = 4 \text{ GB}$
Üçlü dolaylı (triple indirect)	Üç seviye pointer	$1024^3 \times 4 \text{ KB} = 4 \text{ TB}$

Σ Formül

4 KB blok boyutu, 4 bayt pointer ile maksimum dosya boyutu:

$$\begin{aligned}
 F_{\max} &= 12 \cdot 4\text{KB} + \frac{4096}{4} \cdot 4\text{KB} + \left(\frac{4096}{4}\right)^2 \cdot 4\text{KB} + \left(\frac{4096}{4}\right)^3 \cdot 4\text{KB} \\
 &= 48\text{KB} + 4\text{MB} + 4\text{GB} + 4\text{TB} \approx 4\text{TB}
 \end{aligned}$$

David Wheeler'in sözü: “Yazılımdaki her problem başka bir dolaylılık katmanı eklenerek çözülebilir.”

□ Sınav Yorumu

Klasik hesaplama sorusu: “Blok boyutu 1 KB, pointer 4 bayt ise tek dolaylı blok kaç KB’lık dosyayı destekler?”

$$\frac{1024}{4} \times 1 \text{ KB} = 256 \text{ KB}$$

“10 doğrudan + tek dolaylı → maksimum boyut?”

$$10 \times 1 \text{ KB} + 256 \text{ KB} = 266 \text{ KB}$$

Adım adım yazmayı alışkanlık haline getirin.

4.6 Üç Allocation Yönteminin Karşılaştırması

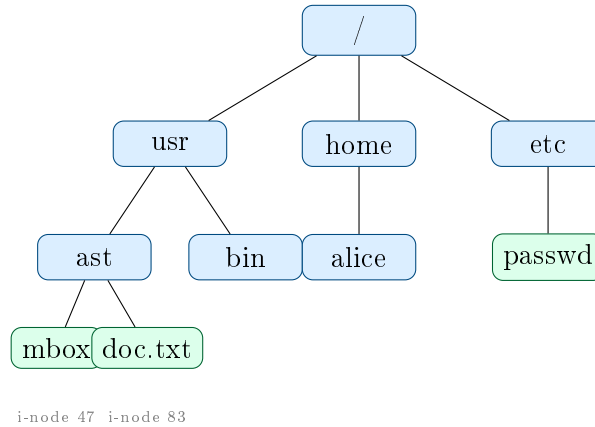
Kriter	Contiguous	Linked	FAT	Indexed (i-node)
Sıralı erişim	Çok iyi	İyi	İyi	İyi
Rastgele erişim	Çok iyi	Kötü	Orta	İyi
Dış parçalanma	Var	Yok	Yok	Yok
İç parçalanma	Var	Var	Var	Var
Blok ekle	Zor	Kolay	Kolay	Kolay
RAM kullanımı	Az	Az	Çok	Orta
Güvenilirlik	Yüksek	Düşük	Orta	Yüksek
Karmaşıklık	Düşük	Düşük	Orta	Yüksek

5 Directory Yapısı ve Uygulaması

5.1 Hiyerarşik Directory Sistemi

★ Önemli Nokta

Directory (Dizin): Dosya adlarını i-node numaraları ile eşleştiren bir veri yapısıdır. Kavramsal olarak bir *hash tablosu* veya *arama ağacı* gibi düşünülebilir.



5.1.1 Path Name Çözümleme

Absolute path (mutlak yol): Kök (/) ile başlar; her zaman kökten itibaren aranır.

Relative path (görelî yol): Geçerli çalışma dizininden başlar.

✓ Örnek

/usr/ast/mbox yolunun çözümlenmesi (UNIX V7):

1. / kök dizini okunur → **usr**'nin i-node numarası bulunur (örn. 6).

2. I-node 6 okunur → `usr` dizin bloğu bulunur → `ast` aranır → i-node 26.
3. I-node 26 okunur → `ast` dizin bloğu → `mbox` aranır → i-node 60.
4. I-node 60 okunur → dosya içeriğine erişim.

Toplam minimum disk erişimi: 6 okuma (3 directory + 3 i-node). Caching varsa bu sayı dramatik düşer.

5.1.2 . ve .. Girişleri

- `.` (nokta): Geçerli dizinin i-node numarasını gösterir.
- `..` (çift nokta): Üst dizinin i-node numarasını gösterir.
- Her yeni dizin oluşturulduğunda çekirdek bu iki girişi otomatik ekler.
- Kök dizinde `..` kökün kendisini gösterir.

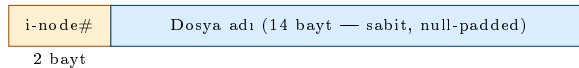
△ Dikkat

Sembolik linkler ve döngüler: `.` ve `..` takibi dizin yapısının ağaç olmasına gerek duymaz. Ancak sembolik linkler döngüye (cycle) neden olabilir! Linux çekirdeği sembolik link takibini belirli bir derinlikle sınırlar (Linux'ta 40 adım).

5.2 Directory Entry Yapıları

5.2.1 Sabit Boyutlu Girişler

Eski UNIX V7: Her directory entry 16 bayttı — 2 bayt i-node numarası + 14 bayt isim.



5.2.2 Değişken Uzunluklu İsimler

Modern sistemler dosya adlarının değişken uzunlukta olmasına izin verir. İki yöntem:

1. **Inline:** Uzunluk alanı + isim direkt entry'de; kayıtlar değişken boyutlu.
2. **Heap yöntemi:** Directory entry sabit boyutlu; isim string ayrı bir heap alanına işaret eder.

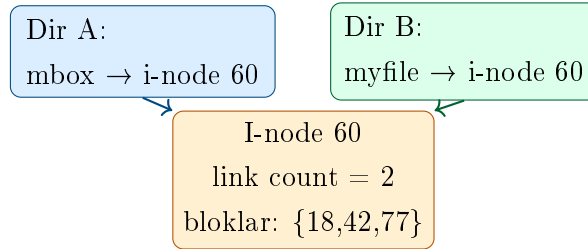
5.3 Shared Files (Hard Links ve Soft Links)

5.3.1 Hard Link

★ Önemli Nokta

Hard link: Farklı bir directory entry'nin aynı i-node'u göstermesidir. I-node'un `link count` (bağlantı sayacı) alanı artırılır.

- Farklı directory entry'ler, aynı fiziksel dosyayı paylaşır.
- Dosya ancak *link count* sıfıra indiğinde gerçekten silinir.
- Hard link yalnızca aynı file system içinde çalışır (i-node numarası filesystem'e özgüdür).
- Dizinelere hard link oluşturmak genellikle yasaktır (döngü riski).



5.3.2 Soft Link (Symbolic Link)

Sembolik link, başka bir dosyanın **yolunu** (path string'ini) içeren özel bir dosyadır. Ayrı bir i-node'u vardır. Farklı filesystem'ler arasında, dizinlere bile link oluşturulabilir. Ancak orijinal dosya silinirse link “kırık” (dangling) kalır.

Özellik	Hard Link	Soft Link (Symlink)
I-node	Aynı i-node paylaşılır	Ayrı i-node (path string içerir)
Filesystem sınırı	Aynı filesystem içinde	Farklı filesystem mümkün
Orijinal silinirse	Veri korunur	Link kırık kalır
Dizin desteği	Genellikle yasak	Mümkün
Link count etkisi	Artar	Artmaz

5.4 Directory Operasyonları

System Call	Açıklama
<code>mkdir</code>	Yeni dizin oluşturur; <code>.</code> ve <code>..</code> ekler
<code>rmdir</code>	Dizini siler (yalnızca <code>.</code> ve <code>..</code> dışında boşsa)
<code>opendir</code>	Dizini okuma için açar
<code>closedir</code>	Açık dizini kapatır
<code>readdir</code>	Sonraki directory entry'yi okur
<code>rename</code>	Yeni isimle link oluşturup eskisini siler
<code>link</code>	Hard link oluşturur
<code>unlink</code>	Hard link kaldırır (link count=0 ise dosyayı siler)

6 File Operasyonları: Açma'dan Kapama'ya

6.1 Dosya Operasyonlarına Genel Bakış

System Call	Ne Yapar?
<code>creat</code> / <code>open</code> (<code>O_CREAT</code>)	Yeni dosya oluşturur
<code>open</code>	Mevcut dosyayı açar; file descriptor döner
<code>close</code>	Dosyayı kapatır
<code>read</code>	Veri okur
<code>write</code>	Veri yazar
<code>lseek</code>	Dosya içi konumu değiştirir
<code>unlink</code>	Dosyayı (hard linkini) kaldırır
<code>stat</code> / <code>fstat</code>	Metadeta okur
<code>chmod</code> / <code>chown</code>	Metadeta değiştirir
<code>rename</code>	Dosyayı yeniden adlandırır

6.2 Dosya Açma (Open)

★ Önemli Nokta

Bir dosya açıldığında:

1. I-node diskten RAM'e okunur (gerekiyorsa) ve **reference count** artırılır.
2. Bir **file table entry** oluşturulur (dosya offset, erişim modu).
3. Bu entry'nin indeksi uygulamaya **file descriptor** olarak döndürülür.

File descriptor, açık bir dosyaya kısa referanstır; tüm OS'lerde bu kavram mevcuttur.

6.3 Dosya Oluşturma (Create)

1. Directory'de eşleşen giriş var mı? Varsa dosya açılır (gerekirse truncate edilir).
2. Yoksa: yeni directory entry yazılır → boş bir i-node bulunup tahsis edilir → **link count** = 1.
3. Directory entry'si i-node numarası 0 ise boş; i-node **link count** = 0 ise serbest.

6.4 Okuma (Read)

1. Mevcut bayt offseti blok numarasına dönüştürülür: $\lfloor \text{offset} / \text{blok_boyutu} \rfloor$
2. İlgili blok diskten okunur (önce buffer cache kontrol edilir).
3. Kullanıcı tamponuna (user buffer) doğru baytlar kopyalanır.
4. Offset güncellenir.
5. **Read-ahead optimizasyonu:** Sıralı erişim tespit edilirse sonraki blok önceden yüklenmeye başlanır.

6.5 Yazma (Write)

1. Bloğun ortasına mı yazılıyor? Önce mevcut blok okunur.
2. Yeni blok ise: freelist'ten blok alınır, i-node'a eklenir.
3. Veri kullanıcı alanından tampona kopyalanır.
4. Tampon **dirty** (kirli) olarak işaretlenir → ilerleyen zamanda diske yazılır.
5. File offset güncellenir.

6.6 Seek

★ Önemli Nokta

`lseek(fd, offset, whence)` yalnızca in-memory dosya offset değerini değiştirir. **Disk kolunu fiziksel olarak hareket ettirmez.** Bu nedenle çok görevli sistemlerde anlamsız ve maliyetli olurdu.

6.7 Dosya Kapatma (Close)

1. I-node'un **reference count**'u düşürülür.
2. File table entry silinir.
3. `link count = 0` **VE** `reference count = 0` ise dosya gerçekten silinir; bloklar freelist'e döner.

△ Dikkat

Açık dosyayı silme: UNIX'te açık bir dosyayı `unlink()` ile silebilirsiniz; ancak dosya fiziksel olarak silinmez — `reference count` sıfıra inene (tüm süreçler kapatana) kadar veriler korunur. Çarpışma (crash) anında bu dosyalar kaybolabilir; `fsck` bu durumu tespit etmeye çalışır.

6.8 Linkleme ve Linklemeyi Kaldırma

`link(old, new):`

- Yeni bir directory entry oluşturulur; i-node numarası mevcut dosyadan alınır.
- I-node'un `link count`'u artırılır.

`unlink(path):`

- `link count` düşürülür.
- Hâlâ > 0 ise: başka isimler var, dosya korunur.
- $= 0$ **VE** `reference count = 0`: tüm bloklar freelist'e iade edilir.

7 Free-Space Yönetimi

7.1 Boş Blok Takibinin Önemi

İşletim sistemi her zaman hangi disk bloklarının boş (free), hangilerinin dolu (allocated) olduğunu bilmek zorundadır. Bunun için iki temel yöntem kullanılır.

7.2 Bağlı Liste (Linked List of Free Blocks)

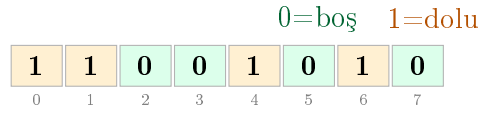
Superblock | Blok A | Blok B ~~next~~

Avantaj: Basit; fazladan alan gerekmez (pointer blok içinde).

Dezavantaj: Boş blok bulmak için liste gezilmeli; büyük listelerde yavaş.

7.3 Bitmap (Bit Haritası)

Her blok için 1 bit: 0 = boş, 1 = dolu.



Σ Formül

Bitmap boyutu:

$$\text{Bitmap boyutu} = \frac{\text{Toplam blok sayısı}}{8} \text{ bayt} = \frac{\text{Disk boyutu}}{\text{Blok boyutu} \times 8} \text{ bayt}$$

Örnek: 200 GB disk, 4 KB blok:

$$\frac{200 \times 10^9}{4096 \times 8} \approx 6.1 \text{ MB}$$

Bu oldukça küçüktür; bitmap genellikle belleğe sığar.

Yöntem	Avantajlar	Dezavantajlar
Bağlı liste	Ek depolama az	Blok bulmak yavaş; pointer bozulması tehlikeli
Bitmap	Hızlı; bitwise işlemlerle blok aranır; RAM'de tutulabilir	Ek alan gerekir (genellikle küçük)

7.4 Disk Kotaları (Disk Quotas)

Çok kullanıcıli sistemlerde her kullanıcının disk kullanımı sınırlandırılabilir. Quota tablosu şunları izler:

- **Soft limit:** Aşılabilir ama uyarı verir.
- **Hard limit:** Asla aşamaz.
- Hem blok sayısı hem i-node sayısı üzerine kota konulabilir.

8 Crash Recovery ve Journaling

8.1 Tutarsızlık Problemi

Bir dosyayı silmek UNIX'te üç ayrı disk yazma işlemi gerektirir:

1. Dosya directory'sinden girdinin kaldırılması.

2. I-node'un serbest i-node havuzuna iade edilmesi.
3. Disk bloklarının serbest blok havuzuna (freelist) iade edilmesi.

Bu üç adım arasında sistem çöker (crash) ya da güç kesilirse disk **tutarsız** (inconsistent) bir durumda kalır. Örneğin:

- Directory girişi silinip i-node serbest bırakılmazsa → blok sahipsiz olur (*missing block*).
- Aynı blok hem bir dosyaya hem freelist'e ait görünürse → *duplicate block*.

★ Önemli Nokta

Temel prensip: Disk her zaman güvenli bir durumda kalacak şekilde yazma sıralı yapılmalıdır. Directory girişini yazmadan önce *metadata* yazılmalıdır.

8.2 fsck — File System Consistency Check

fsck (UNIX) ve **scandisk/chkdisk** (Windows), önyükleme zamanında dosya sistemini kontrol eden araçlardır. Temiz kapatma (clean shutdown) bayrağı *set* değilse çalışır.

8.2.1 Blok Tutarlılığı Kontrolü

fsck iki tablo tutar:

- **Tablo 1:** Her blok kaç dosyaya ait? (i-node'lardan sayılır)
- **Tablo 2:** Her blok freelist'te kaç kez görünüyor?

Durum	Açıklama	Çözüm
T1=1, T2=0	Normal: blok bir dosyaya ait	—
T1=0, T2=1	Normal: blok freelist'te	—
T1=0, T2=0	<i>Missing block</i> : kayıp blok	Freelist'e ekle
T1=0, T2=2	<i>Duplicate free block</i>	Bir kopyayı listeden çıkar
T1=2, T2=0	<i>Duplicate data block</i> : iki dosya aynı bloğa işaret	Bir kopyaya farklı blok ver

(a) Tutarlı T1=1, T2=0	(b) Kayıp blok T1=0, T2=0	(c) Dup. free T1=0, T2=2	(d) Dup. data T1=2, T2=0
------------------------------	---------------------------------	--------------------------------	--------------------------------

□ Sınav Yorumu

fsck sınav soruları:

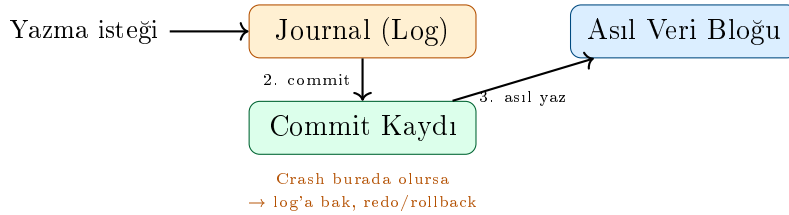
- “fsck nasıl çalışır?” → İki tablo, blok tutarlılığı ve i-node link count kontrolü.
- “Bir blok hem bir dosyaya ait hem freelist’te görünüyorsa ne olur?” → fsck freelist girişini kaldırır.
- “Bir blok iki farklı dosyaya aitse fsck ne yapar?” → Bir kopyasını alır, alternatif bloka kopyalar.

8.3 Log-Structured File Systems ve Journaling

★ Önemli Nokta

Journaling (Kayıt Günlüğü): Veriyi doğrudan *üzerine yazmak* yerine, yapılacak değişiklik önce **journaling alanına** (log) yazılır. Yalnızca bu kayıt commit edildikten sonra asıl veri güncellenir.

Faydası: Sistem her an tutarlı; crash sonrasında log’a bakılarak yarım kalan işlem ya *redo* (tekrarlanır) ya da *rollback* (geri alınır) edilir. fsck gibi uzun bir tarama gerekmez.



8.3.1 Modern Disk Yazma Problemleri

Modern diskler dahili **yazma cache**'i (2–16 MB) kullanır ve yazmaları *seek optimizasyonu* için yeniden sıralar. Güç kesintisinde cache'deki veri kaybolabilir. Bu yüzden:

- Yazma sıralama garantisi **yalnızca flush/sync komutlarıyla** verilebilir.
- Journaling'in etkinliği disk firmware'inin flush'a uymasına bağlıdır.
- Kötü sektörlerin diskin kendisi tarafından yeniden konumlandırılması seek sürelerini tahmin edilemez kılar.

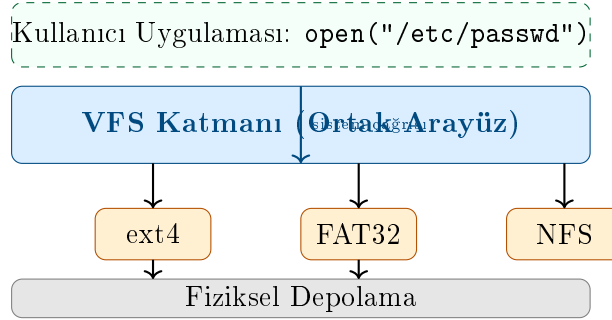
9 Virtual File System (VFS)

9.1 VFS Nedir ve Neden Gereklidir?

Modern işletim sistemleri aynı anda birçok farklı dosya sistemi tipini destekler: ext4, NTFS, FAT32, tmpfs, NFS... Kullanıcı uygulaması hangi dosya sistemine eriştiğini *bilmek zorunda değildir*; `open()`, `read()`, `write()` her zaman aynı biçimde çalışır.

★ Önemli Nokta

VFS (Virtual File System): Farklı dosya sistemi türlerini tek bir arayüz arkasına gizleyen **soyutlama katmanıdır**. İşletim sistemi içinde dosya sistemi bağımsız bir katman olarak yer alır; somut (concrete) dosya sistemi uygulamaları bu katmanın altındadır.



9.2 VFS'in Çalışma Prensibi

VFS her open dosya için bir **vnode** (virtual node) oluşturur. Vnode içinde dosya sistemi tipine özgü fonksiyon pointer'ları (*metotlar*) bulunur:

Listing 2: VFS — vnode operasyonları (basitleştirilmiş)

```

1 struct vnode_ops {
2     int (*vop_read)(vnode_t *, uio_t *);
3     int (*vop_write)(vnode_t *, uio_t *);
4     int (*vop_lookup)(vnode_t *, char *name, vnode_t **result)
5     ;
6     int (*vop_create)(vnode_t *, char *name, vattr_t *);
7     int (*vop_remove)(vnode_t *, char *name);
8     // ...
9 };
  
```

✓ Örnek

/etc/passwd (ext4) ve /mnt/usb/foto.jpg (FAT32) okuma senaryosu:

- Her iki dosya da `read()` ile açılır.
- VFS, her dosya için farklı `vop_read` fonksiyonunu çağırır (biri ext4 implementasyonu, diğeri FAT32 implementasyonu).
- Kullanıcı uygulaması bu farkı *görmez*.

□ Sınav Yorumu

“VFS’in amacı nedir?” sorusu sıkça çıkar. Yanıt: Farklı dosya sistemi tiplerini tek bir arayüz arkasına saklamak; uygulamaları dosya sistemi türünden bağımsız kılmak. Object-oriented tasarımdaki *interface* / *polymorphism* kavramına benzetme yapılabilir.

10 File System Backup

10.1 Yedeklemenin Amacı

Yedekleme (backup) iki temel tehlikeye karşı yapılır:

1. **Felaket kurtarma (disaster recovery):** Disk arızası, yangın, su baskını.
2. **Hata kurtarma (stupidity recovery):** Yanlışlıkla silinen dosyalar.

10.2 Tasarım Kararları

Soru	Seçenekler
Neyi yedekle?	Tüm sistem mi, yalnızca belirli dizinler mi?
Ne zaman yedekle?	Tam (full) yedek mi, artımlı (incremental) yedek mi?
Sıkıştırma?	Bant alanı tasarrufu vs. ek CPU yükü
Çevrimiçi vs. çevrimdışı	Yedek alınırken dosyalar değişiyor mu?
Güvenlik	Bantlar nerede saklanıyor? Şifreli mi?

10.3 Fiziksel Dump vs. Mantıksal Dump

Özellik	Physical Dump	Logical Dump
Yöntem	Blok 0’dan itibaren birebir kopyala	Dizin ağacını özyinelemeli gez; dosyaları kaydet
Boş bloklar	Kopyalanır (israf)	Atlanır
Artımlı yedek	Desteklenmez	Desteklenir (değişen dosyalar)
Geri yükleme	Yalnızca aynı disk geometrisine	Herhangi bir sisteme
Hız	Çok hızlı	Daha yavaş

10.4 Mantıksal Dump Algoritması

★ Önemli Nokta

4 Aşamalı Algoritma (Bitmap Tabanlı):

1. **Faz 1:** Tüm değiştirilmiş dosyaları ve *tüm* dizinleri (değiştirilmiş olsun olmasın) işaretle.
2. **Faz 2:** İçinde değiştirilmiş dosya olmayan dizinlerin işaretini kaldır.
3. **Faz 3:** İşaretleli tüm dizinleri bantlara dök (*dump*).
4. **Faz 4:** İşaretleli tüm dosyaları bantlara dök.

Her dosya/dizin için bitmap'te birkaç bit tutulur: değişti mi? Yedeklenecek mi? Yedeklendi mi?

△ Dikkat

Dizinlerin değişmemiş olsa bile yedeklenmesi gerekir çünkü geri yükleme sırasında dosya sisteminin tam izin yapısının yeniden kurulması gerekir.

10.5 Geri Yükleme Algoritması

1. Disk üzerinde boş bir dosya sistemi oluştur.
2. En son tam (full) yedekten geri yükle: önce dizinler, sonra dosyalar.
3. Her artımlı (incremental) yedeği sırasıyla uygula.
4. Geri yükleme tamamlandıktan sonra free blok listesini yeniden oluştur.

Özel durumlar:

- İki dizinde hard link ile paylaşılan dosyalar yalnızca *bir kez* geri yüklenmeli; linkleme tekrar kurulmalı.
- Artımlı yedekte silinmiş dosyaları temsil etmek için *whiteout* girişleri kullanılabilir.

10.6 UNIX Dump Seviyeleri

UNIX dump seviyesi 0 tam yedekti; seviye 1 son seviye 1'den bu yana değişen her şeyi; seviye 2 son seviye 1'den bu yana değişenleri içerir.

✓ Örnek

Tipik strateji:

- Pazar: Seviye 0 (tam yedek).
- Pazartesi–Cumartesi: Seviye 1 (değişen dosyalar).
- Her ayın 1'i: Seviye 0 ofsite gönderilir.

Geri yükleme: en son tam yedek + en son artımlı yedek.

11 File System Performansı: Caching ve Optimizasyon

11.1 Temel Sorun: Disk vs. Bellek Hızı

Disk erişimi bellek erişiminden **milyonlarca kat daha yavaştır**. Bu farkı kapatmak için üç temel teknik kullanılır:

Teknik	Açıklama
Caching (Önbellekleme)	Sık erişilen bloklar RAM'de tutulur
Block read-ahead	Sıralı erişim tespit edilirse sonraki blok önceden okunur
Disk arm motion azaltma	I-node ve veri blokları aynı cylinder grubunda tutulur

11.2 Buffer Cache

★ Önemli Nokta

Buffer cache (tampon önbelleği): Diskten okunan blokların RAM'de tutulduğu havuzdur. Aynı blok tekrar istenirse diskten okunmaz; önbellekten karşılanır. Yazma işlemleri de önce cache'e yapılır (*write-back caching*).

Buffer cache tipik olarak bir **hash tablosu** + **LRU (Least Recently Used) listesi** ile yönetilir:

- Hash tablosu: blok numarası → cache girişi hızlı bulma.
- LRU listesi: en uzun süre kullanılmayan bloğu çıkarma (evict).

11.2.1 Değiştirilmiş LRU (Modified LRU)

Bazı bloklar (i-node blokları, superblock) kısa sürede iki kez erişilmez; önbelleğin başına alınmaları anlamsızdır. Geliştirilmiş LRU iki faktörü dikkate alır:

1. **Yakın zamanda tekrar kullanılacak mı?** (Erişim olasılığı)
2. **Dosya sisteminin tutarlılığı için kritik mi?** (I-node, süper blok: önbellekten en kısa sürede diske yazılmalı)

△ Dikkat

Kritik metadata blokları (superblock, i-node) cache'de çok uzun süre kalmaktır tehlikeli! Crash olursa ve bunlar henüz diske yazılmamışsa tutarsızlık oluşur. Bu yüzden bazı sistemler bu blokları *write-through* (hemen diske yaz) politikası ile yönetir.

11.3 Block Read-Ahead (Blok Önceden Okuma)

Dosyaya sıralı erişim tespit edildiğinde OS, kullanıcı henüz istemeden sonraki bloğu asenkron olarak okumaya başlar. Blok hazır olduğunda kullanıcı program bekleme yapmadan veriyi alır.

11.4 Disk Kol Hareketini Azaltma (Cylinder Groups)

★ Önemli Nokta

Cylinder group (silindir grubu): BSD Unix'in benimsediği yöntemde disk, silindir gruplarına bölünür. Her silindir grubunda:

- İlgili i-node'ların bir kısmı,
- Bu i-node'lara ait boş blok listesi,
- Veri alanı bulunur.

Yeni bir dosya oluşturulduğunda bloklar aynı silindir grubundan tahsis edilmeye çalışılır → **locality of reference** → disk kolu çok ileriye gitmez → seek süresi düşer.

11.5 Blok Boyutu Seçimi

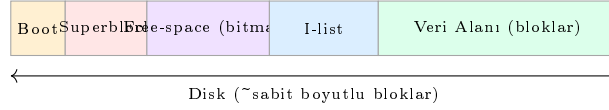
Blok boyutu, hem disk veri transfer hızını hem alan verimliliğini etkiler:

- **Büyük blok:** Yüksek transfer hızı, ancak küçük dosyalar için iç parçalanma artar.
- **Küçük blok:** Alan verimliliği yüksek, ancak büyük dosya için çok disk erişimi.

Pratikte 4 KB, iyi bir denge noktası olarak benimsenmiştir. Bazı sistemler büyük dosyalar için **fragment** (512 bayt alt birim) kullanır; dosyanın son parçası tam blok yerine fragment üzerinde saklanır.

12 File System Bileşenleri: Superblock, I-list, Data Area

12.1 Tipik Disk Düzeni



12.2 Boot Record ve Superblock

- **Boot record (MBR):** Diskin en başında yer alır; BIOS tarafından önyükleme sırasında okunur. Disk bölüm (partition) bilgisini de içerir.
- **Superblock:** Dosya sistemi parametrelerini barındırır:
 - Toplam blok sayısı, blok boyutu.
 - I-list başlangıç adresi ve büyüklüğü.
 - Boş blok/i-node sayısı.
 - **“Clean shutdown” bayrağı:** Temiz kapanışta set; önyüklemede clear edilir. Bayrak set değilse fsck çalıştırılır.

△ Dikkat

Superblock bozulursa tüm dosya sistemi kaybolur! Bu nedenle modern dosya sistemleri (ext2/3/4) superblock'un birkaç kopyasını disk üzerinin farklı yerlerinde saklar.

12.3 I-list (I-node Listesi)

- Eskiden superblock'un hemen ardından tek blok dizisi olarak tutulurdu.
- Modern BSD/Linux'ta **silindir gruplarına** dağıtılmıştır; her silindir grubu kendi i-list parçasını içerir.
- I-node numarası = dizinin indeksi; sabit boyutlu kayıtlar.

12.4 Veri Alanı (Data Area)

- Dosya içeriklerinin saklandığı asıl alan.
- BSD Unix'te iki boyut: **bloklar** (4–8 KB) ve **fragmentler** (512 B); büyük dosyalar blok, küçük dosyaların son parçaları fragment kullanır.

13 UNIX V7 Dosya Sistemi — Adım Adım

13.1 UNIX V7 Directory Entry

UNIX V7'de directory entry son derece sade:

- 2 bayt i-node numarası
- 14 bayt dosya adı (null-padded, sabit boyut)

13.2 Dosyaya Erişim: /usr/ast/mbox

✓ Örnek

Adım adım blok erişimleri:

1. I-node 1 (kök dizin /): okunur → veri bloğu okunur → **usr** → i-node 6 bulunur.
2. I-node 6 (/usr): okunur → veri bloğu okunur → **ast** → i-node 26 bulunur.
3. I-node 26 (/usr/ast): okunur → veri bloğu okunur → **mbox** → i-node 60 bulunur.
4. I-node 60 (mbox dosyası): okunur → blok adresleri → dosya içeriği okunur.

Özet: n bileşenli bir yol için $2n$ disk erişimi gerekir (her bileşen için 1 i-node + 1 veri bloğu). Caching bu sayıyı büyük ölçüde düşürür.

Σ Formül

Yol çözümleme disk erişimi (caching yoksa):

$$\text{Erişim sayısı} = 2 \times (\text{yol bileşeni sayısı})$$

Tekli dolaylı blok erişimi:

Dosyanın N . bloğuna ($N > 12$) erişmek için doğrudan erişime ek olarak 1 ekstra disk I/O (indirect block okuma).

Çift dolaylı: 2 ekstra; Üçlü dolaylı: 3 ekstra.

14 Windows FAT Dosya Sistemi

14.1 FAT'ın Tarihçesi ve Sürümleri

Sürüm	Bit Genişliği	Max Bölüm Boyutu	Max Dosya
FAT12	12 bit	32 MB	32 MB
FAT16	16 bit	2 GB	2 GB
FAT32	28 bit (4 GB)	8 TB	4 GB
exFAT	32 bit	128 PB	128 PB

14.2 FAT Yapısı

- **Root directory:** Sabit boyutlu; kök dizin girdilerini tutar.

- **8.3 dosya isimleri:** 8 karakter ad + 3 karakter uzantı (DOS uyumluluğu).
- **Metadata:** Directory entry içinde saklanır (UNIX i-node'u gibi ayrı yapı yok).
- **Directory → FAT table entry:** Her dosya FAT tablosunda zincirleme pointer'larla temsil edilir.

14.3 Uzun Dosya Adı Desteği (VFAT)

Windows 98 ile eklendi. Eski DOS uyumluluğunu korumak için:

1. Uzun adın tanınan kısmından geçerli bir 8.3 adı türetilir.
2. Gerçek uzun ad, **geçersiz görünen (invalid)** sahte directory girişleri olarak önceki slot'lara yazılır; DOS bu girdileri yok sayar.
3. Gerçek kısa ad girişinin silinip silinmediğini kontrol etmek için *checksum* tutulur.

15 ISO 9660 (CD-ROM) Dosya Sistemi

ISO 9660 CD-ROM için standart dosya sistemidir. İlk 16 blok OS veya önyükleme için ayrılmıştır; ardından Primary Volume Descriptor (CD-ROM hakkında genel bilgi) gelir.

- Dosya isimleri büyük harf; 8 seviye dizin derinliği.
- **Rock Ridge uzantısı:** POSIX öznitelikler, sembolik linkler, büyük/küçük harf.
- **Joliet uzantısı:** Uzun dosya adları, Unicode karakter seti, 8 seviyeden derin dizin, uzantılı dizin adları.

16 Uygulamalı Mini Problemler

16.1 Problem 1: FAT Bellek Hesabı

Problem

100 GB disk, 512 bayt blok boyutu, her FAT entry 4 bayt olsun. FAT tablosu için gereken RAM miktarı nedir?

Çözüm:

$$\text{Blok sayısı} = \frac{100 \times 10^9}{512} \approx 195,312,500$$

$$\text{FAT RAM} = 195,312,500 \times 4 \text{ bayt} \approx 781 \text{ MB}$$

Bu çok büyük! Bu yüzden FAT sistemleri küçük blok boyutlarından kaçınır; FAT32 tipik olarak 4–32 KB blok kullanır.

16.2 Problem 2: I-node Hesabı — Dosya Boyutu

Problem

Blok boyutu 1 KB, pointer boyutu 4 bayt. 10 doğrudan blok + 1 tek dolaylı blok + 1 çift dolaylı blok kullanan i-node ile maksimum dosya boyutu nedir?

Çözüm:

$$p = \frac{1024}{4} = 256 \text{ pointer/blok}$$

$$F = 10 \times 1 \text{ KB} + 256 \times 1 \text{ KB} + 256^2 \times 1 \text{ KB} = 10 + 256 + 65536 \text{ KB} = 65802 \text{ KB} \approx 64.3 \text{ MB}$$

16.3 Problem 3: Yol Çözümleme Erişim Sayısı

Problem

Cache yok. /a/b/c/dosya.txt yolunu açmak kaç disk erişimi gerektirir?

Çözüm:

- Kök (/): i-node oku + veri bloğu oku = 2
- a: i-node + veri bloğu = 2
- b: i-node + veri bloğu = 2
- c: i-node + veri bloğu = 2
- dosya.txt: i-node oku = 1
(veri bloğu henüz okunmadı)

$$\text{Toplam} = 4 \times 2 + 1 = 9 \text{ disk erişimi}$$

16.4 Problem 4: Bitmap Boyutu

Problem

500 GB disk, 4 KB blok boyutu. Free-space bitmap için kaç bayt gerekir?

$$\text{Blok sayısı} = \frac{500 \times 10^9}{4096} \approx 122,070,313$$

$$\text{Bitmap} = \lceil 122,070,313/8 \rceil \approx 15.3 \text{ MB}$$

16.5 Problem 5: fsck Senaryo Analizi

Problem

fsck taraması sırasında Blok 42 için: Tablo 1 = 0, Tablo 2 = 0. Ne anlama gelir? fsck ne yapar?

Yanıt: Blok 42 ne bir dosyaya ait, ne de freelist'tedir. Bu *kayıp blok (missing block)* durumudur. Disk alanı harcanmış ama kullanılmıyor. fsck bu bloğu freelist'e ekler.

17 UNIX File System Operasyonları — Kod Düzeyinde

17.1 Tipik Dosya İşlem Akışı

Listing 3: Basit dosya okuma/yazma örneği

```

1  #include <fcntl.h>
2  #include <unistd.h>
3
4  int main() {
5      int fd;
6      char buf[4096];
7      ssize_t n;
8
9      /* Dosyayı aç */
10     fd = open("/etc/passwd", O_RDONLY);
11     if (fd < 0) { perror("open"); return 1; }
12
13     /* Oku */
14     while ((n = read(fd, buf, sizeof(buf))) > 0) {
15         write(STDOUT_FILENO, buf, n); /* stdout'a yaz */
16     }
17
18     /* Kapat */
19     close(fd);
20     return 0;
21 }
```

17.2 Rename Operasyonunun İnceliği

Listing 4: rename() — neden kernel içinde yapılmalı?

```

1  /* Kullanıcı alanında yapılıyadı (HATALI/GÜVENSİZ): */
2  link("eski_isim", "yeni_isim"); /* Hard link oluştur */
3  unlink("eski_isim"); /* Eski linki kaldır */
4  /* ^ Bu iki adım arasında crash olursa iki isim var! */
5
6  /* Doğrusu: rename() system call -- kernel garantisi */
7  rename("eski_isim", "yeni_isim"); /* Atomik (ya hep ya hiç)
   */
```

17.3 Dizin Oluşturma ve Silme

Listing 5: Dizin operasyonları

```

1 #include <sys/stat.h>
2 #include <dirent.h>
3
4 /* Dizin oluştur -- kernel . ve .. ekler */
5 mkdir("/home/user/yeni_dizin", 0755);
6
7 /* Dizin listele */
8 DIR *dir = opendir("/home/user");
9 struct dirent *entry;
10 while ((entry = readdir(dir)) != NULL) {
11     printf("%s_(_inode:_%lu)\n", entry->d_name, entry->d_ino);
12 }
13 closedir(dir);
14
15 /* Dizini sil -- yalnızca . ve .. varsa */
16 rmdir("/home/user/yeni_dizin");

```

18 Karşılaştırma Tabloları Özeti

18.1 Dosya Sistemleri Karşılaştırması

Özellik	FAT32	ext4	NTFS	ISO 9660
Allocation	Linked (FAT)	Extent/i-node	B-tree/MFT	Contiguous
Max dosya	4 GB	16 TB	16 EB	4 GB
Journaling	Yok	Var	Var	Yok (ROM)
Hard link	Yok	Var	Var	Yok
İzin sistemi	Basit	POSIX	ACL+POSIX	Yok (opsiyonel)
Metadata konumu	Directory	I-node	MFT kaydı	Directory

18.2 Crash Recovery Yöntemleri Karşılaştırması

Yöntem	Nasıl Çalışır?	Avantaj	Dezavantaj
Yazma sıralaması	Önce metadata, sonra veri	Basit	Tutarlılık garantisi zayıf
fsck	Önyüklemede blok/i-node taraması	Var olan sistemlerle uyumlu	Büyük disklerde çok yavaş
Journaling	Önce log, sonra asıl yazma	Hızlı kurtarma; güçlü tutarlılık	Ek yazma yükü
Copy-on-Write	Değişiklik yeni bloğa yazılır	Çok güçlü (ZFS, btrfs)	Karmaşık; ek alan

19 Terimler Sözlüğü (Glossary)

Terim	Tanım
Allocation	Bir dosyaya disk bloklarının atanması işlemi.
Bitmap	Her disk bloğu için 1 bit tutan serbest alan veri yapısı.
Block	Dosya sisteminin kullandığı okuma/yazma birimi (genellikle 4 KB).
Buffer cache	Diskten okunan blokların RAM’de tutulduğu önbellek havuzu.
Contiguous allocation	Dosya bloklarının disk üzerinde ardışık sıralanması.
Crash recovery	Sistem çökmesi sonrasında dosya sistemini tutarlı hale getirme.
Cylinder group	BSD’de disk bölümü; i-node + veri + freelist yerel gruplandırması.
Directory	Dosya adı → i-node numarası eşlemelerini tutan veri yapısı.
Double indirect block	Pointer’lara pointer tutan dolaylı blok (2. katman).

Terim	Tanım
External fragmentation	Boş alan yeterli ama parçalı olduğundan tahsis edilemeyen durum.
FAT (File Allocation Table)	Linked allocation'ın tüm pointer'ları bellekteki tabloda tutan sürümü.
File descriptor	Açık dosyayı temsil eden tamsayı; process'in file table indeksi.
File system	Dosya adlarını fiziksel ortamın bloklarına eşleyen OS soyutlaması.
Fragment	BSD Unix'te son blok parçaları için 512 B alt birim.
fsck	UNIX'te dosya sistemi tutarlılığı denetçisi.
Hard link	Aynı i-node'u gösteren birden fazla directory entry.
I-list	Diskte tüm i-node'ların bulunduğu dizi.
I-node (index node)	Dosyanın tüm metadata'sını ve blok adreslerini tutan yapı.
Internal fragmentation	Tahsis edilen son blokta kullanılmayan boş kısım.
Journaling	Değişiklikleri önce log'a yazıp sonra uygulayan crash-safe yöntem.
Link count	Bir i-node'a işaret eden hard link sayısı.
Linked allocation	Her blokun sonraki bloğun adresini taşıdığı zincirleme yapı.
Logical dump	Değişmiş dosyaları seçerek yedekleme yöntemi.
MBR (Master Boot Record)	Diskin en başındaki önyükleme + bölüm bilgisi alanı.
Metadata	Dosya hakkındaki veri: izinler, boyut, tarihler, blok adresleri.
Physical dump	Disk bloklarını birebir yedekleme; boş bloklar dahil.
Read-ahead	Sıralı erişimde bir sonraki bloğun önceden okunması.
Sector	Diskin donanım düzeyi en küçük I/O birimi (512 B).
Single indirect block	Pointer dizisi tutan dolaylı blok (1. katman).

Terim	Tanım
Soft link (symlink)	Başka dosyanın yolunu içeren özel dosya; ayrı i-node.
Superblock	Dosya sistemi parametrelerini tutan kritik blok.
VFS (Virtual File System)	Farklı dosya sistemlerini tek arayüzle sunan OS soyutlama katmanı.
WORM	Write Once Read Many — bir kez yazılabilen disk türü.

20 Sınav Cevap Şablonları

20.1 “Allocation yöntemlerini karşılaştırın” — Model Yanıt

Model Yanıt Şablonu

Dosya sistemi tasarımında üç temel allocation yöntemi vardır:

- 1. Contiguous allocation:** Her dosya ardışık bloklar kaplar. Avantajı hem sıralı hem rastgele erişimin hızlı olması, uygulamanın basit olmasıdır. Dezavantajı dış parçalanmaya yol açması ve dosya büyütmeyi zorlaştırmasıdır. CD-ROM gibi sabit boyutlu dosyalarda idealdir.
- 2. Linked allocation:** Her blok sonraki bloğun adresini taşır. Dış parçalanma yoktur, dosya büyütme kolaydır. Ancak rastgele erişim çok yavaştır ve bozuk bir pointer tüm dosyayı mahvedebilir. FAT bu yöntemi geliştirerek pointer'ları bellekteki tabloya taşır.
- 3. Indexed allocation (i-node):** Tüm blok adresleri tek bir veri yapısında (i-node) toplanır. Hem sıralı hem rastgele erişim iyidir; dış parçalanma yoktur. Çok katmanlı dolaylı bloklar ile çok büyük dosyalar desteklenir. UNIX/Linux bu yöntemi benimser.

20.2 “I-node’u açıklayın” — Model Yanıt

Model Yanıt Şablonu

I-node (index node), UNIX dosya sistemlerinde bir dosyanın tüm metadata'sını ve disk blok adreslerini tutan sabit boyutlu yapıdır. İçeriği:

- Dosya türü ve izinleri (**mode**)
- Sahip (uid, gid)
- Dosya boyutu ve blok sayısı
- Erişim/değişiklik/durum tarihleri (atime, mtime, ctime)
- Hard link sayısı (link count)
- Doğrudan + dolaylı disk blok adresleri

I-node numarası, i-list adlı dizideki konumunu belirtir. Directory entry yalnızca “dosya adı → i-node numarası” eşlemesini tutar; bu sayede birden fazla isim aynı i-node’u gösterebilir (hard link). I-node yalnızca dosya açıkken RAM’e yüklenir; kapatıldığında diske geri yazılır.

20.3 “Journaling nedir?” — Model Yanıt**Model Yanıt Şablonu**

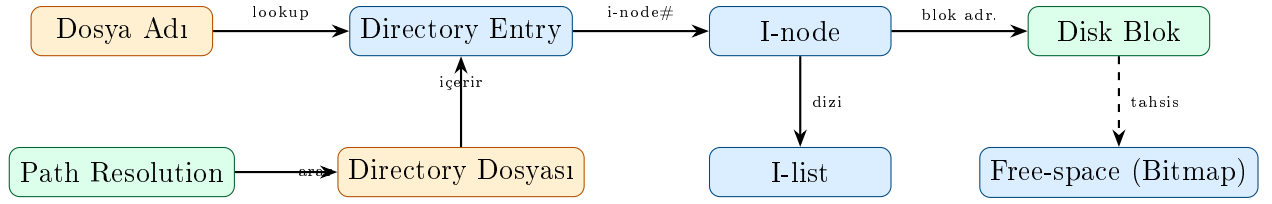
Journaling, crash recovery için kullanılan bir dosya sistemi tekniğidir. Temel fikir: değişiklikleri doğrudan asıl veri üzerine yazmak yerine önce bir “journal” (günlük) alanına kaydetmek, ardından asıl bloğu güncellemektir.

Çalışma akışı:

1. Yapılacak işlem journal’a yazılır (begin + record).
2. Commit kaydı journal’a yazılır.
3. Asıl veri bloğu güncellenir.
4. Journal temizlenir.

Crash olduğunda: commit varsa işlem tamamlanır (redo); commit yoksa işlem geri alınır (rollback). Bu sayede fsck gibi uzun bir disk taraması gereksiz hale gelir.

21 Kavramsal Bağlantı Haritası



★ Önemli Nokta

Dosyaya erişimin tam zinciri:

Dosya adı $\xrightarrow{\text{Directory lookup}}$ i-node# $\xrightarrow{\text{I-list indexleme}}$ I-node (metadata+blok adr.) $\xrightarrow{\text{Blok haritası}}$ Disk

Bu zincirin her adımı için cache (VFS inode cache, directory cache, buffer cache) devreye girerek disk erişimlerini minimize eder.

22 Final Revizyon: 25 Kritik Sınav Noktası

Sınav Öncesi Kontrol Listesi

- Dosya sistemi için 3 temel gereksinim:** Çok büyük veri, kalıcılık, eş zamanlı erişim.
- UNIX dosya yapısı:** Bayt dizisi (*byte sequence*); OS içeriği yorumlamaz.
- Contiguous allocation avantajı / dezavantajı:** Hızlı erişim / dış parçalanma.
- Linked allocation rastgele erişim problemi:** N . bloğa ulaşmak N adım gerektirir.
- FAT = Linked allocation + Bellekteki tablo:** Disk okumadan pointer takibi; ama büyük RAM tüketir.
- FAT RAM formülü:** Disk boyutu/Blok boyutu $\times$ Entry boyutu (bayt).
- I-node = Index node:** Tüm metadata + blok adresleri; directory'de yalnızca isim $\rightarrow$ i-node# var.
- Çok kademeli dolaylı blok:** Direct(12) + Single(1024) + Double(1024²) + Triple(1024³) blok.
- I-node numarası:** I-list dizisinin indeksi; hard link $\rightarrow$ aynı i-node, farklı directory entry.

10. **Hard link vs. soft link:** Hard $\rightarrow$ aynı i-node, aynı filesystem, link count artar; Soft $\rightarrow$ ayrı i-node, path string, farklı filesystem mümkün.
11. **Link count sıfırlanana kadar dosya silinmez.** Açık dosya unlink $\rightarrow$ kapanınca gerçek silme.
12. **Directory entry:** Ad + i-node numarası (UNIX V7'de 14+2 bayt).
13. **. ve ..** $\rightarrow$ Kendi ve üst dizinin i-node numaraları; kernel otomatik ekler.
14. **Yol çözümleme (cache yok):** Her bileşen için 2 disk erişimi (i-node + veri bloğu).
15. **Bitmap boyutu:** Blok sayısı/8 bayt; genellikle RAM'e sığar.
16. **fsck iki tablo tutar:** Her blok kaç dosyaya ait (T1) + freelist'te kaç kez (T2). T1=T2=0 $\rightarrow$ kayıp blok.
17. **Journaling prensip:** Önce log $\rightarrow$ commit $\rightarrow$ asıl yazma. Crash $\rightarrow$ redo veya rollback.
18. **Superblock:** Dosya sistemi parametreleri + clean shutdown bayrağı. Bozulursa yıkıcı.
19. **VFS:** Farklı dosya sistemi türlerini tek arayüzle sunan soyutlama. Vnode + metot pointer'ları.
20. **Buffer cache LRU:** Sık erişilen bloklar RAM'de; dirty bloklar ilerleyen zamanda diske yazılır.
21. **Write-through kritik metadata:** Superblock ve i-node blokları geciktirilmeden yazılmalı; crash riski.
22. **Cylinder group:** I-node + veri bloğu yerel gruplandırması $\rightarrow$ seek süresi azalır (locality).
23. **Fiziksel dump vs. mantıksal dump:** Fiziksel: birebir, hızlı, artımlı yok; Mantıksal: seçici, artımlı mümkün.
24. **Mantıksal dump 4 fazı:** (1) Değişmiş dosya + tüm izinleri işaretler; (2) Boş izinleri kaldır; (3) Dizinleri dök; (4) Dosyaları dök.
25. **FAT32 max dosya boyutu 4 GB** (28-bit blok adresi); exFAT bu sınırı aşar.

23 Ek: Önemli Formüller Özeti

Σ Formül

Temel Formüller

$$N_{\text{blok}} = \left\lceil \frac{\text{Dosya boyutu}}{\text{Blok boyutu}} \right\rceil \quad \text{İç parçalanma} = N_{\text{blok}} \times \text{Blok boyutu} - \text{Dosya boyutu}$$

$$\text{FAT RAM} = \frac{\text{Disk boyutu}}{\text{Blok boyutu}} \times \text{Entry boyutu}$$

$$\text{Bitmap boyutu (bayt)} = \left\lceil \frac{\text{Toplam blok sayısı}}{8} \right\rceil$$

$$p = \frac{\text{Blok boyutu}}{\text{Pointer boyutu}} \quad (\text{bir dolaylı bloktaki pointer sayısı})$$

$$F_{\text{max}} = d \cdot B + p \cdot B + p^2 \cdot B + p^3 \cdot B$$

(d : doğrudan blok sayısı, B : blok boyutu, p : pointer sayısı/blok)

$$\text{Yol erişimi (cache yok)} = 2 \times \text{Bileşen sayısı} + 1 \quad (\text{son i-node})$$

Başarılar!

CSE 312 — İşletim Sistemleri — File Systems Ders Notu